EECS 498 · AASE · FALL 2026

How Aider works

LAB01 · DESIGN YOUR PAIR-PROGRAMMER

September 14–15, 2026

Today: understand, then design

Minutes	Work
0–27	Follow one request through Aider
27–37	Diagnose the model, context, and harness
37–40	Hackathon briefing
40–55	Your project, design artifacts, and grading
55–65	Claude Code and OpenClaw comparison
65–110	Draft, sketch, and review a first increment

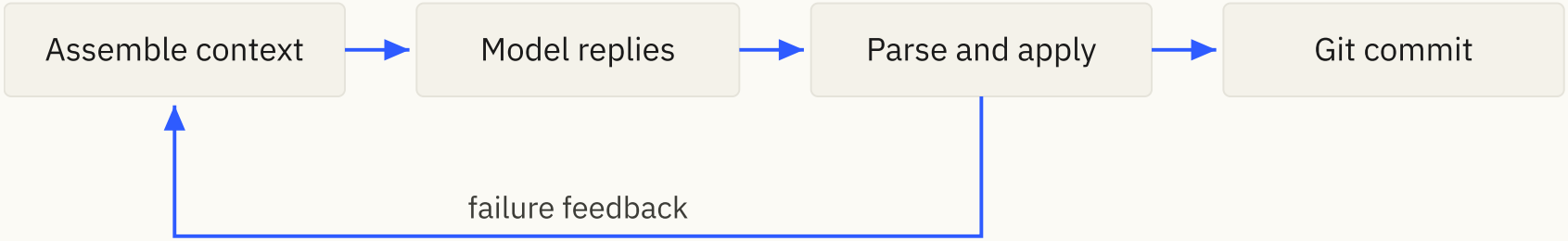
One familiar request

```
Add a --priority flag to taskr's add command.
```

You already met this request in the practice lessons.

Today we follow an illustrative diff-mode turn, from selected files to a commit.

Who performs each action?



The harness prepares context and acts on the reply. The model supplies text.

The human chooses the task and evaluates the result.

1. Assemble the model input

The request combines instructions, selected file contents, relevant history, and your new message.

Aider can also supply a repository map and edit-format examples.

Context is selected information. A file outside it may exist on disk without being available in full to the model.

Choose editable and reference files

```
/add taskr/cli.py taskr/task.py taskr/store.py
/read-only specs/priority.md
/tokens
```

Editable files can be changed. Read-only files explain the contract.

A linked document is not automatically loaded into context.

The repository map is a summary

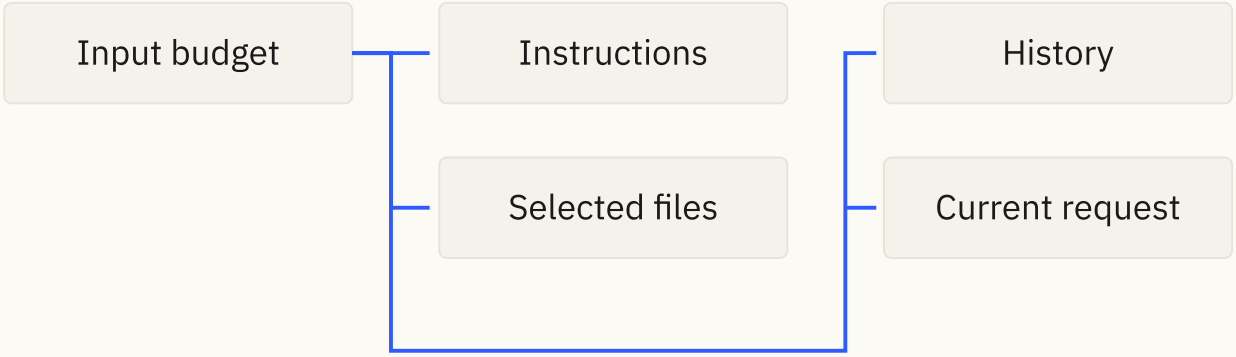
```
taskr/store.py
  Store.add(title, tags) -> Task

taskr/task.py
  Task: id, title, tags
```

Aider ranks useful definitions and references within a token budget.

A signature can suggest where to look. It does not supply the function body.

Context has a budget



More history leaves less room for current evidence.

If the necessary input cannot fit, the tool needs an explicit policy.

2. The reply is still text

```
taskr/cli.py
<<<<<<< SEARCH
add.add_argument("title")
=======
add.add_argument("title")
add.add_argument("--priority", default="normal")
>>>>>>> REPLACE
```

This is a proposed replacement, not evidence that priority works.

3. Parse structure from the reply

A parser identifies a path, a SEARCH section, and replacement text.

It must also report malformed blocks without crashing.

Parsing and validation answer different questions: is this a block, and is this block applicable here?

4. Applying an edit changes state

A useful proposal still needs a match against the actual file.

Aider can use fallback matching and can retain successful edits when other blocks fail.

Your Stage 1 contract requires exact, unambiguous matches and all-or-nothing application.

5. Failure can become new input

```
Proposed edit
    → match failed
    → report the failure to the model
    → receive another proposal
```

Aider also supports lint/test feedback cycles when configured.

A feedback loop can exist without general model-selected tool use.

6. Git records the accepted change

A commit gives a concrete before/after boundary.

Undo needs rules about ownership and changes made afterward.

Git is useful recovery infrastructure. It does not undo every side effect a program can cause.

Explain the request back to us

For the priority feature, identify:

1. Which information enters context?
2. Which program interprets the reply?
3. What could appear correct while still being wrong?
4. What evidence would establish the feature works?

Diagnose before re-prompting

Observation	Investigate
Code calls an interface that does not exist	Missing context or incorrect design
Valid-looking block cannot match	Stale text or invalid generated edit
Tests pass but the feature is incomplete	Weak acceptance criteria or tests
A request times out	Endpoint configuration, service, or limits

Same model, different harness

A bare chat client can suggest code.

Aider adds repository context, an edit protocol, and actions on files.

Better results may come from better context and checking, even when the model is unchanged.

Practice: choose the next action

The model adds `--priority`. The command runs. After restarting the program, every task has normal priority.

What do you inspect before asking the model to try again?

Write one missing acceptance criterion and name a test that would catch it.

Hackathon 1: September 24

Two hours in the course browser workspace.

A new feature prompt is revealed in the room and applied to your own project.

Submit during the session. Keep a runnable increment available beforehand.

The event has separate grading. Ask staff early about conflicts or accommodations.

Your project starts with a design

You receive the assignment packet and development tooling.

You design the application, write its specification, and use Aider to implement it in increments.

There is no supplied conversation loop or required module layout.

The application you are building

A terminal assistant for **questions and approved edits to selected text files**.

```
/files                → list selected files
/add greet.py         → include this file in model requests
What does greet do?   → stream an answer about the selected code
Change Hello to Hi.   → validate the reply and preview a diff
```

You run `aider` to build it. You implement `bin/assistant` to run it.

A proposal is not a file change

Suppose the model proposes changing `Hello` to `Hi` in `greet.py`.

Point in the interaction	File contents	Git outcome
Diff shown, awaiting approval	Still `Hello`	No new commit
User approves	Now `Hi`	One owned edit commit
User declines instead	Still `Hello`	No new commit
User undoes the approved edit	Back to `Hello`	Restore the parent HEAD

One invalid block cancels the entire proposal, including valid blocks.

All seven features remain

Feature	Required behavior
F1	Conversation, budget handling, streamed replies
F2	Add, drop, list, and clear context
F3	Current file contents rendered consistently
F4	A tested edit-format prompt
F5	Parsing with useful errors
F6	Exact edits, complete preview, explicit approval
F7	One commit per accepted edit set, guarded undo

YAML config, any compatible endpoint

```yaml
base_url: https://api.example.edu/v1
model: course-model-id
api_key_env: ASSISTANT_API_KEY
context_budget: 8000
timeout_seconds: 60
temperature: 0.2
```

You implement configuration, the endpoint client, and validation.

Staff requirements, student decisions

We specify	You design
Observable behavior and safety rules	Responsibilities and state ownership
API and configuration contract	Internal interfaces and error flow
Acceptance scenarios and rubric	Tests, fixtures, implementation increments
Semester direction	Where later capabilities can enter

Two levels of specification

System spec: behavior, architecture, interfaces, failure handling, verification, and rationale.

Increment specs: scoped context, ordered tasks, and a testable result for each change.

Commit the initial design before application implementation. Revise it when evidence changes a decision.

Three diagrams explain your design

Diagram	Question it answers
Components and data flow	Who owns state and crosses each boundary?
Request sequence	Who does what during one proposed edit?
Edit lifecycle	When may an edit be approved, rejected, or undone?

Keep initial versions in git. Update the final diagrams to describe what you built.

40 points: specification and diagrams

Criterion	Points
Requirements and acceptance criteria	10
Architecture and interface design	10
Aider implementation specs	10
Diagrams: components 4, sequence 3, lifecycle 3	10

Completeness and usefulness matter. Document length does not.

40 points: behavior and verification

Criterion	Points
Required functionality, scored by feature	20
Behavioral tests	8
Safety and recovery tests	7
Fake integration 3, live evidence 2	5

A correct failing test can earn test credit while exposing an unfinished feature.

20 points: evidence and reconciliation

Criterion	Points
Spec-first history	5
Deliberate Aider use	5
Diagnosis and design reconciliation	5
Operating instructions	3
Model disclosure and evidence index	2

The model rule includes the spec

Use the permitted Stage 1 models for drafting, critique, diagrams, code, and tests.

The 9B is recommended; the 4B is permitted. Record secondary models too.

Any compatible hosting service is allowed. A different provider does not authorize a different model.

From a first increment to December

Start with one small, verifiable result. Continue through all requirements.

Analyze adds model-selected tools and a loop. Create hardens the agent and adds memory and another way to reach it.

Keep one repository and history. Refactor when needed; preserve phase snapshots.

What changes with Claude Code?

Aider workflow	General agentic CLI workflow
Human frames a bounded edit task	Human supplies a broader task
Selected files and a repo map guide context	Tools can discover and read files
Edit/repair cycles follow harness rules	Model can choose tools and next steps
Human checks progress between tasks	Permissions and stop rules bound autonomy

The same priority request, later

An agentic CLI may search for argument parsing, read storage code, edit both, and run tests before returning.

Each tool result becomes evidence for the next decision.

Who controls the next step, and what stops a bad one?

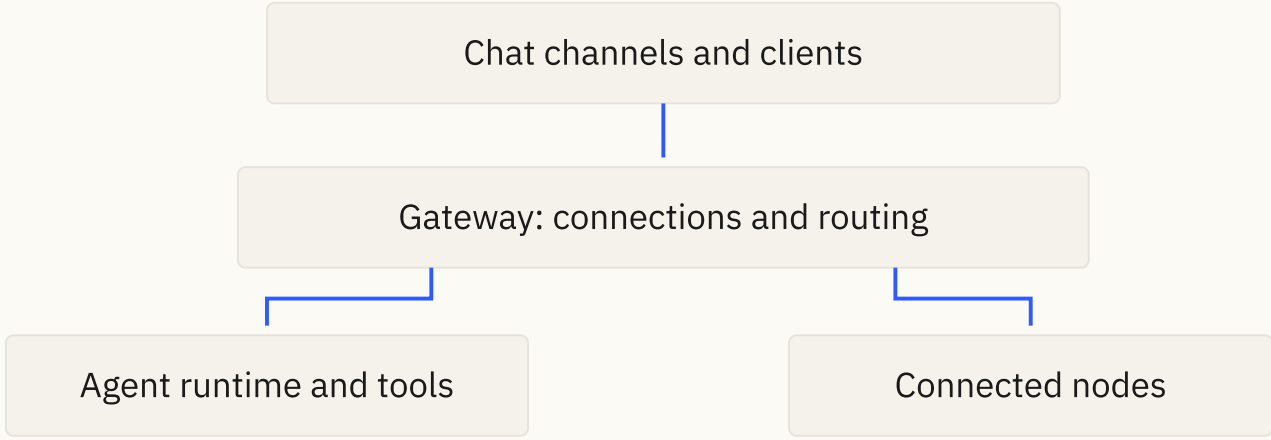
Instructions still need a harness

Project instructions provide persistent guidance.

Skills package instructions for particular tasks. Hooks attach checks to events. Tool integrations expose actions and results.

None of these removes the need for clear contracts and verification.

OpenClaw adds a gateway



A gateway gives an agent ways to receive requests and reach connected capabilities.

Design for the next boundary

Where could a model-selected tool enter your current control flow?

Could the endpoint client be replaced without rewriting state management?

Could a web interface use the same application behavior as the terminal?

Work block: 45 minutes

Minutes	Your result
0–8	Read the packet; confirm development tooling
8–23	Interpret requirements; sketch responsibilities
23–35	Write a first increment with a checkable result
35–43	Exchange a focused review; ask staff about blockers
43–45	Commit the draft and record your next decision

Review a decision, not the polish

Ask your partner:

- What must be true when this increment is done?
- Which decision would the model still have to guess?
- What test would catch a plausible mistake?

Record the question and your response. Bring assignment ambiguities to staff.

Before you implement

Finish the initial system design and diagrams.

Use a fresh permitted-model session to review them. Commit the design and the first increment spec.

Then build, test, and revise one increment at a time.